Data Networks

BitTorent, A peer to peer file sharing system

Department of Electrical Engineering
Sharif University of Technology

Instructor: Dr. Mohammad Reza Pakravan

Submitted by:

Amirali Yazdiani - 402102765

Date:
Aug 2026

Contents

1 Torrent File Generation

To make the configuration process user friendly and human readable, the initial configurations for the torrents were written in JSON format. This allows for easy modification of the target files and piece lengths without dealing with raw binary data.

Listing 1: Single File Torrent Example

```
{
  "announce": "http://127.0.0.1:8000/announce",
  "info": {
    "name": "Transferring_These_Files/sample3.png",
    "length": 1455741,
    "piece length": 262144,
    "pieces": ""
  }
}
```

Listing 2: Multi File Torrent

```
{
  "announce": "http://127.0.0.1:8000/announce",
  "info": {
    "name": "Transferring_These_Files",
    "piece length": 262144,
    "pieces": "",
    "files": [
      {
        "length": 1116449,
        "path": ["sample1.png"]
      },
      {
        "length": 1455741,
        "path": ["sample2.png"]
      }
    ]
  }
}
```

Using the `torrent_generator.py` script, these JSON files are parsed, the actual physical files are read, and their SHA-1 hashes are computed for each piece. The script then generates the final `.txt` file encoded in the Bencode format.

Why Bencode? While JSON is easy to read, the BitTorrent protocol mandates Bencoding. This is because Bencoding explicitly supports arbitrary binary data (like the concatenated SHA-1 hashes in the `pieces` field) and ensures a deterministic serialization order. This determinism is critical because any slight change in formatting would alter the `info_hash`, completely breaking peer discovery in the network.

2 Tracker Web Service

The Tracker acts as the central directory of the BitTorrent ecosystem. It does not store or transfer any of the actual files. Its sole responsibility is to track which peers are currently in the swarm and facilitate their discovery.

In this implementation, the tracker operates as a multithreaded HTTP server. It maintains a dictionary (`swarm_db`) where the keys are the torrent `info_hash`es and the

values are lists of active peers (IP and Port). When a peer connects to the `/announce` endpoint, the tracker handles three specific events:

- **Started:** Adds the peer to the swarm database or updates its port if it already exists.
- **Completed:** Marks the peer as a "seeder".
- **Stopped:** Gracefully removes the peer from the database so other peers do not attempt to connect to a dead node.

The tracker then responds with a bencoded dictionary containing the swarm statistics and the list of available peers.

3 Peer Client Operation

The peer client is the core component responsible for downloading and seeding files. The execution flow follows a strict, step by step procedure:

1. **Initialization:** The peer generates a unique 20 byte identifier (starting with `-AA0001-`) to represent itself in the swarm.
2. **Port Binding:** It attempts to open a TCP listening socket starting at port 6881. If the port is in use, it sequentially tries the next ports up to 6889.
3. **Tracker Communication:** The peer sends an HTTP GET request to the tracker's announce URL to register itself and retrieve a list of other active peers.
4. **Handshake:** Upon receiving peer addresses, it initiates direct TCP connections and performs the 68 byte BitTorrent handshake to verify protocol compatibility and the `info_hash`.
5. **Data Exchange:** Peers exchange bitfield messages to declare which pieces they have. The client then sends `INTERESTED` messages and requests file pieces sequentially using `REQUEST` messages, saving them directly to a local buffer and verifying their SHA-1 hashes upon receipt.

The BitTorrent Handshake Protocol

The BitTorrent protocol mandates a strict 68 byte handshake as the first message transmitted when two peers establish a connection. This message is critical for verifying protocol compatibility and ensuring both peers are interested in the same torrent file.

Figure 1: Structure of the 68-byte BitTorrent Handshake Message

As illustrated in Figure 1, the 68 bytes are structured into five distinct sections:

1. **pstrlen (1 Byte):** The string length of the protocol identifier. For the standard BitTorrent protocol, this value is always `19` (0x13 in hexadecimal).
2. **pstr (19 Bytes):** The string identifier of the protocol. It is strictly encoded as the string `"BitTorrent protocol"`.
3. **reserved (8 Bytes):** Eight reserved bytes set to zero (`0x00`). These bytes are reserved for future extensions.
4. **info_hash (20 Bytes):** The 20 byte SHA-1 hash of the `info` key from the bencoded metainfo (`.torrent`) file. This verifies that both peers are attempting to transfer the exact same swarm data.
5. **peer_id (20 Bytes):** A unique 20 byte string randomly generated by the client upon startup to identify itself within the swarm.

4 Centralized Logger

To properly monitor the asynchronous nature of peer to peer networks, a custom `logger.py` was implemented. Standard console outputs are insufficient when multiple threads and clients are running concurrently.

The logger dynamically creates isolated log files for both the Tracker and each individual Peer. It categorizes every event (handshakes, ping successes, errors) and records them with precise timestamps in a structured format containing a header (metadata) and a body (tabulated event history).

5 Web Based Dashboard (AI Assisted)

To better comprehend the real time topological changes and traffic within the network, a graphical dashboard was developed using HTML, JavaScript, and the Vis.js library.

The dashboard connects to the tracker's API to visually map the swarm and animate network events such as `PING`, `PONG`, `PIECE`, and `REQ` messages, providing a highly intuitive understanding of the underlying protocol mechanics.

It should be noted that this graphical interface was developed with the assistance of Artificial Intelligence.

6 Unimplemented Features & Future Work

While the core file sharing mechanics are fully functional, some advanced BitTorrent specifications were omitted in this implementation:

- **Rarest First Algorithm:** The current system fetches pieces sequentially rather than prioritizing the rarest pieces available in the swarm.
- **Choking/Unchoking Logic:** Advanced bandwidth optimization algorithms are not fully enforced.

GitHub Repository & Parallel Download Branch:
The entire source code is available in the following GitHub repository:
https://github.com/amirali83/Project

An attempt was made to implement simultaneous multi peer downloading (Swarm downloading). This work is located in a separate branch named `parallel_download`. However, this branch is currently unstable and suffers from synchronization and state management bugs, thus it is not included in the main implementation.

7 Step by Step Execution Guide

To execute the project and observe the file sharing process locally, follow these exact steps:

1. **JSON Configuration:** It is highly recommended **not** to modify the `torrent1.json` and `torrent2.json` files. They are already strictly linked to the existing local files and correctly demonstrate both single file and multi file architectures. Adding a third torrent would require hardcoding logic changes within the peer script.

2. **Generate Bencoded Torrents:** Open a terminal and run the generator script to convert the JSON configurations into the final bencoded `.txt` files required by the system:

   ```
   python torrent_generator.py
   ```

3. **Start the Tracker:** In a new terminal, execute the tracker server. Once running, you can open a web browser and navigate to `http://127.0.0.1:8000/dashboard` to access the graphical visualization.

   ```
   python Tracker.py
   ```

4. **Initialize Peers:** Open multiple new command prompt (CMD) windows. Run the peer client in each window, providing a unique ID index:

   ```
   python peer.py 1
   python peer.py 2
   ```

   Upon execution, each peer creates its own isolated directory. If this directory already exists from a previous run, the peer resumes its state; otherwise, it starts completely empty.

5. **Internal Command Line:** Once the peer is running, an internal prompt (`peer>`) appears. You can use the following commands:

   - `download 1, 2`: Instructs the peer to start fetching the corresponding torrent file.
   - `exit`: Gracefully shuts down the peer and notifies the tracker to remove it from the swarm.

8 Practical Execution and Dashboard Demonstration

To illustrate the real time capabilities of the system, this section provides a step by step visual demonstration of a file transfer session, observed directly through the graphical dashboard.

8.1 Initialization

First, the tracker server is initiated by executing `Tracker.py`. At this stage, the swarm database is completely empty. Navigating to the HTML dashboard reflects a solitary Tracker node with no active swarms or traffic.

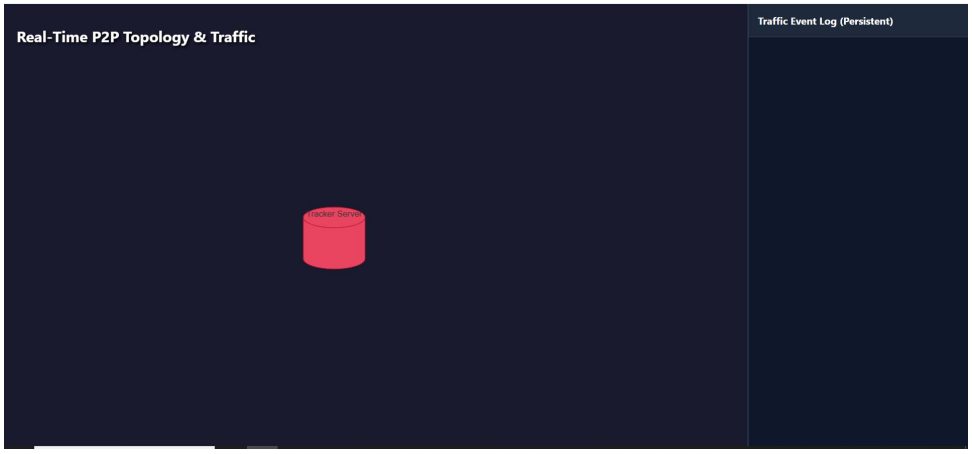


Figure 2: Initial state of the dashboard before any peers join the network.

8.2 Seeder Registration

Next, we execute `peer.py 1`. This peer's workspace already contains the complete files defined in our first torrent. Upon startup, the client scans its local directory, verifies the integrity of the files, and automatically registers with the tracker as a Seeder. The dashboard dynamically updates to show the torrent swarm and the newly joined peer.

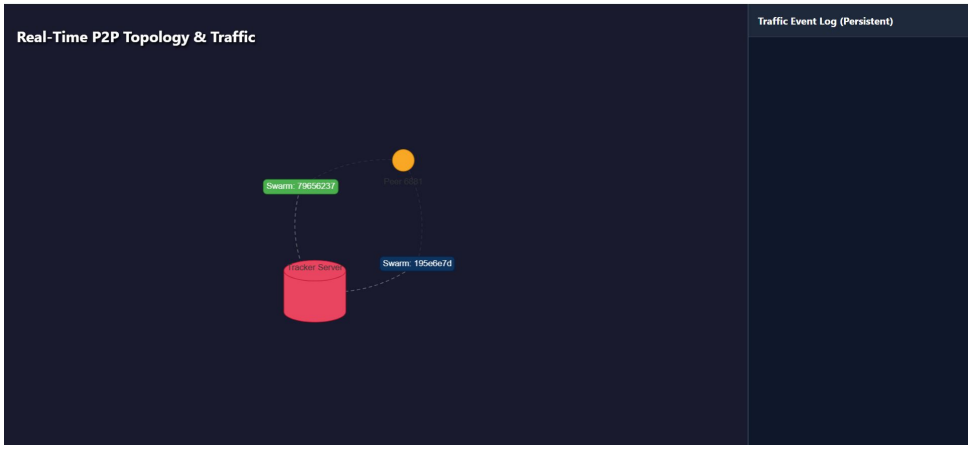
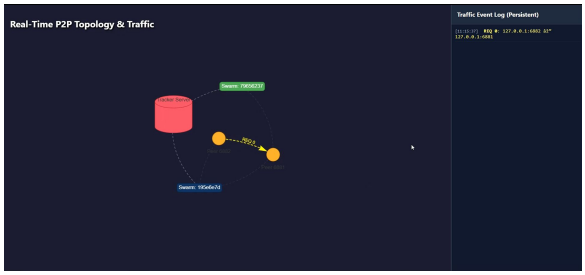


Figure 3: The dashboard displays Peer 1 joined as a seeder for the torrent swarm.

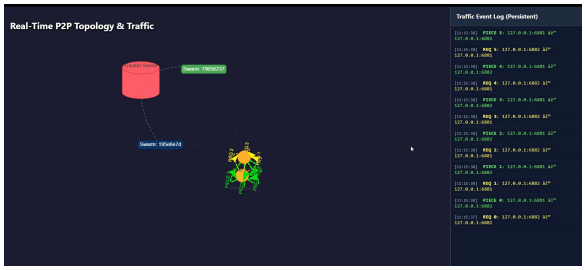
8.3 Leecher Joining and Data Transfer

To simulate a download, a new client is launched by executing `peer.py 3`. Because this ID index was not used previously, the script generates a fresh, empty workspace directory named `workspace_-AA0001-000000000003`.

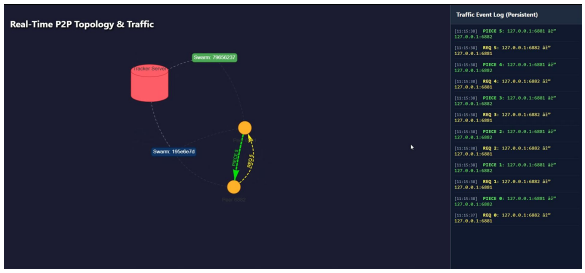
Within the internal terminal of Peer 3, the command `download 1` is issued to initiate the download process. The tracker introduces Peer 3 to Peer 1, handshakes are exchanged, and the piece transfer begins. The dashboard captures this rapid exchange of `REQ` and `PIECE` messages seamlessly.



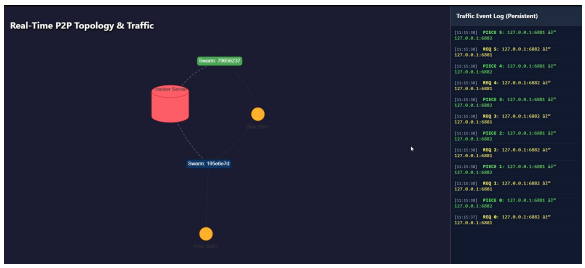
(a) Initiating data transfer: Peer 3 requesting the first piece (REQ 0).



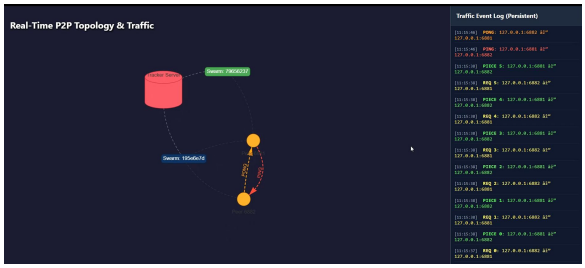
(b) Continuous exchange: Multiple REQ and PIECE messages being transmitted.



(c) Nearing completion: Transmission of the final REQ and PIECE.



(d) Download complete: The peer successfully acquired all file pieces.



(e) Swarm maintenance: Periodic PING and PONG heartbeats between peers.

Figure 4: Chronological progression of the data transfer phase and subsequent swarm network maintenance.

8.4 Graceful Disconnection

Once the transfer is complete, or if a user decides to leave the swarm, the `exit` command is executed in the peer's terminal. The peer transmits a `stopped` event to the tracker. The tracker successfully deregisters the peer from its internal database, and the node is subsequently cleared from the dashboard topology.

9 Wireshark and Network Observation

To verify the application level implementation and observe the network stack behavior, Wireshark was used to capture the local loopback traffic.

9.1 Tracker Communication

By filtering the traffic on `tcp.port == 8000`, the HTTP communication between the peers and the tracker is isolated. As shown in Figure 5, the peer successfully sends an HTTP `GET` request containing the URL encoded `info_hash`, `peer_id`, and `port`. The tracker processes this event and responds with an HTTP `200 OK` containing the bencoded dictionary of the swarm state.

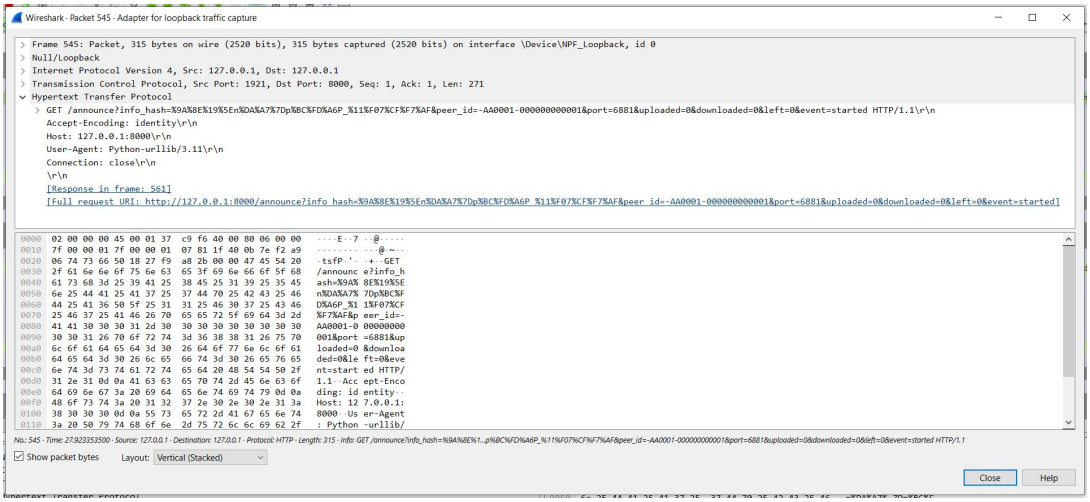
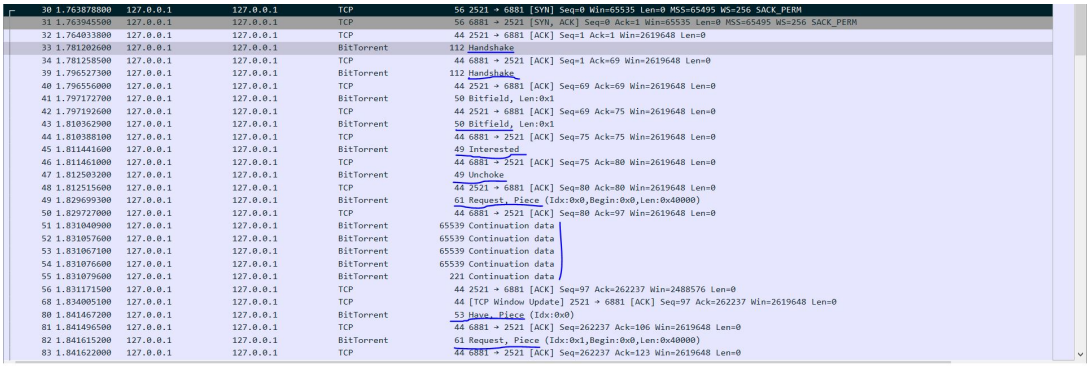


Figure 5: Wireshark capture of the HTTP GET request sent to the Tracker's /announce endpoint.

9.2 Peer-to-Peer TCP Fragmentation

Filtering the capture for `tcp.port >= 6881` reveals the direct communication between peers. A critical observation regarding the BitTorrent protocol is the distinction between application level pieces and network level packets.

In this project, the `piece length` is configured to 256 KiB. Because a single Ethernet MTU is typically 1500 bytes, the operating system's network stack must fragment a single `PIECE` message into multiple sequential TCP packets. Figure 6 demonstrates this behavior, showing numerous `[TCP segment of a reassembled PDU]` packets. The peer implementation handles this seamlessly by utilizing a continuous `recv_all` buffer loop to reconstruct the complete payload independent of TCP boundaries.



30	1.763878800	127.0.0.1	127.0.0.1	TCP	56 2521 → 6881 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
31	1.763945500	127.0.0.1	127.0.0.1	TCP	56 6881 → 2521 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
32	1.764033800	127.0.0.1	127.0.0.1	TCP	44 2521 → 6881 [ACK] Seq=1 Ack=1 Win=2619648 Len=0
33	1.781202600	127.0.0.1	127.0.0.1	BitTorrent	112 Handshake
34	1.781258500	127.0.0.1	127.0.0.1	TCP	44 6881 → 2521 [ACK] Seq=1 Ack=69 Win=2619648 Len=0
39	1.796527300	127.0.0.1	127.0.0.1	BitTorrent	112 Handshake
40	1.796556000	127.0.0.1	127.0.0.1	TCP	44 2521 → 6881 [ACK] Seq=69 Ack=69 Win=2619648 Len=0
41	1.797172700	127.0.0.1	127.0.0.1	BitTorrent	50 Bitfield, Len:0x1
42	1.797192600	127.0.0.1	127.0.0.1	TCP	44 2521 → 6881 [ACK] Seq=69 Ack=75 Win=2619648 Len=0
43	1.810362900	127.0.0.1	127.0.0.1	BitTorrent	50 Bitfield, Len:0x1
44	1.810388100	127.0.0.1	127.0.0.1	TCP	44 6881 → 2521 [ACK] Seq=75 Ack=75 Win=2619648 Len=0
45	1.811441600	127.0.0.1	127.0.0.1	BitTorrent	49 Interested
46	1.811461000	127.0.0.1	127.0.0.1	TCP	44 6881 → 2521 [ACK] Seq=75 Ack=80 Win=2619648 Len=0
47	1.812503200	127.0.0.1	127.0.0.1	BitTorrent	49 Unchoke
48	1.812515600	127.0.0.1	127.0.0.1	TCP	44 2521 → 6881 [ACK] Seq=80 Ack=80 Win=2619648 Len=0
49	1.829699300	127.0.0.1	127.0.0.1	BitTorrent	61 Request, Piece (Idx:0x0,Begin:0x0,Len:0x40000)
50	1.829727000	127.0.0.1	127.0.0.1	TCP	44 6881 → 2521 [ACK] Seq=80 Ack=97 Win=2619648 Len=0
51	1.831040900	127.0.0.1	127.0.0.1	BitTorrent	65539 Continuation data
52	1.831057600	127.0.0.1	127.0.0.1	BitTorrent	65539 Continuation data
53	1.831067100	127.0.0.1	127.0.0.1	BitTorrent	65539 Continuation data
54	1.831076600	127.0.0.1	127.0.0.1	BitTorrent	65539 Continuation data
55	1.831079600	127.0.0.1	127.0.0.1	BitTorrent	221 Continuation data
56	1.831171500	127.0.0.1	127.0.0.1	TCP	44 2521 → 6881 [ACK] Seq=97 Ack=262237 Win=2488576 Len=0
68	1.834005100	127.0.0.1	127.0.0.1	TCP	44 [TCP Window Update] 2521 → 6881 [ACK] Seq=97 Ack=262237 Win=2619648 Len=0
80	1.841467200	127.0.0.1	127.0.0.1	BitTorrent	53 Have, Piece (Idx:0x0)
81	1.841496500	127.0.0.1	127.0.0.1	TCP	44 6881 → 2521 [ACK] Seq=262237 Ack=106 Win=2619648 Len=0
82	1.841615200	127.0.0.1	127.0.0.1	BitTorrent	61 Request, Piece (Idx:0x1,Begin:0x0,Len:0x40000)
83	1.841622000	127.0.0.1	127.0.0.1	TCP	44 6881 → 2521 [ACK] Seq=262237 Ack=123 Win=2619648 Len=0

Figure 6: Observation of TCP packet fragmentation for a large BitTorrent PIECE transmission.